



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes: Gil Valentín Oswaldo

Grupo: 6CV1 **Fecha:** 13/04/2026

Actividad: Ejercicio 2.6 Demultiplexacion TCP vs UDP

Ejercicio: Demultiplexación en Servidores TCP vs UDP

Se requiere implementar dos servidores:

- Un servidor TCP concurrente
- Un servidor UDP

Ambos deben recibir mensajes con el siguiente formato:

<ID_CLIENTE>:<OPERACIÓN>:<VALOR>

Donde:

ID_CLIENTE → identificador único (string)

OPERACIÓN → una de: SQR, CUBE, NEG

VALOR → número entero

Ejemplo:

A1:SQR:4

B2:NEG:-7

La operación SQR calcula el cuadrado del número; la operación CUBE obtiene el cubo del valor; y la operación NEG cambia el signo del número, convirtiendo valores positivos en negativos y viceversa.

Parte 1: Servidor TCP (Demultiplexación explícita)

El servidor debe:

Aceptar múltiples clientes simultáneamente

Implementar concurrencia usando UNA opción:

- select()
- threads

Requerimientos:

Mantener una estructura que asocie cada cliente con su socket:

clientes[socket] = ID_CLIENTE

Procesar cada mensaje recibido y responder:

<ID_CLIENTE>:<RESULTADO>

Manejar eventos de:

- solicitud de conexión
- desconexión de clientes
- mensajes inválidos (ERROR)

Parte 2: Servidor UDP (Demultiplexación implícita)

El servidor debe:

Usar un solo socket

Atender múltiples clientes sin establecer conexión

Requerimientos:

Identificar clientes usando:

clientes[(IP, puerto)] = ID_CLIENTE

Procesar mensajes igual que en TCP

Responder al cliente correcto usando la información de la capa de transporte

Parte 3: Comparativa de Demultiplexación

Analizar y responder:

1. Identificación de clientes

¿Cómo identifica el servidor a cada cliente en TCP?

Primero bloqueamos a los clientes para evitar que 2 clientes entren a la función de asignar las ID al mismo tiempo.

Después aumentamos el contador de los clientes conectados y agregamos la palabra “Cliente_” más el número de conexión global en el servidor y lo añadimos a la lista de clientes conectados.

¿Cómo lo hace en UDP?

Recibiendo el Datagrama del cliente, la IP y el Puerto obtenida mediante la función `recvfrom()` es el único mecanismo para demultiplexar y saber a quién debe enviarle la respuesta, dado que solo hay un socket.

2. Evidencia en código

Coloca un fragmento de código que muestre:

¿Dónde ocurre la demultiplexación en TCP?

La separación física/de red ocurre al aceptar el socket y pasarlo a un nuevo hilo (el SO demultiplexa el tráfico hacia el hilo correcto).

```
while True:
    conn, addr = server_sock.accept()
    # Por cada cliente, se crea un nuevo hilo que lo atiende
    hilo_cliente = threading.Thread(target=manejar_cliente, args=(conn, addr))
    hilo_cliente.daemon = True
    hilo_cliente.start()
```

La demultiplexación lógica ocurre cuando en el código se asocia ese socket físico (conn) con una IP (Cliente_1) y procesa los mensajes basándose en esa IP.

```
with clientes_lock:
    contador_clientes += 1
    id_asignado = f"Cliente_{contador_clientes}"
    clientes[conn] = id_asignado # Lo registramos inmediatamente

print(f"\n[+] CONEXIÓN: {addr} ha sido registrado como '{id_asignado}'")

# Le enviamos un mensaje especial al cliente diciéndole quién es
mensaje_bienvenida = f"ASSIGN:{id_asignado}\n"
conn.sendall(mensaje_bienvenida.encode('utf-8'))

# --- FASE DE COMUNICACIÓN NORMAL ---
with conn:
    while True:
        try:
            data = conn.recv(1024)
            if not data:
                break

            mensaje = data.decode('utf-8').strip()
            id_recibido, resultado = procesar_mensaje(mensaje)

            # Armamos la respuesta según el formato <ID_CLIENTE>:<RESULTADO>
            respuesta = f"{id_recibido}:{resultado}\n"
            conn.sendall(respuesta.encode('utf-8'))
```

¿Dónde ocurre en UDP?

Obtenemos los datos del cliente en donde se usa la función `recvfrom()`

```
try:
    while True:
        #Recibimos el datagrama y la dirección del remitente (IP, Puerto)
        data, addr = server_sock.recvfrom(1024)

        mensaje = data.decode('utf-8').strip()
        print(f"[Paquete de {addr}] -> {mensaje}")

        #Procesamos Los datos
        id_recibido, resultado = procesar_mensaje(mensaje)

        #Armamos la respuesta
        id_respuesta = id_recibido if id_recibido else "DESCONOCIDO"
        respuesta = f"{id_respuesta}:{resultado}\n"

        #Enviamos la respuesta de vuelta directamente al remitente (addr)
        server_sock.sendto(respuesta.encode('utf-8'), addr)
```

3. Pregunta crítica

Si dos clientes envían el mismo ID_CLIENTE, ¿cómo los distingue realmente el servidor en TCP y en UDP?

En TCP:

En el caso de mi código no podría, debido a que el servidor es el que asigna las ID a los clientes, pero en un caso donde el cliente es el que genera la ID se podría distinguir por el puerto y la IP de donde proviene el cliente. Además que se trabajan por hilos distintos.

En UDP:

Al ser un protocolo sin conexión, no existen los sockets dedicados. Sin embargo, el servidor los distingue porque cada datagrama recibido mediante la función `recvfrom()` devuelve siempre la dupla (IP, Puerto) de origen. Si dos clientes envían el mismo ID, el servidor sabrá diferenciarlos gracias a que sus puertos de origen (o sus IPs) serán forzosamente distintos.

Parte 4 Análisis

Responder (máx. 5 líneas cada uno):

¿Cuál implementación fue más sencilla? ¿por qué?

Considero que el de UDP, dado que solo se trabajaba con un solo socket y que en UDP no era necesario realizar un Handshake ni enviar mensajes de confirmación como en el caso de TCP.

¿Qué ventajas observas en TCP?

En TCP se tiene un mayor control lógico y de seguridad al momento de realizar la implementación. Al ser orientado a la conexión, permite arquitecturas donde el servidor tiene el control total (como asignar los IDs a los clientes tras un saludo inicial). Además, el uso de hilos permite una excelente concurrencia, atendiendo a múltiples clientes simultáneamente.

¿Qué ventajas observas en UDP?

Como UDP es un protocolo sin conexión, no tiene que realizar el handshake ni enviar mensajes de confirmación, lo que lo hace mucho más rápido a nivel de red y más sencillo de implementar. Esto es sumamente útil para ahorrar recursos del sistema, ya que evita la sobrecarga computacional que TCP genera al tener que crear hilos y mantener el estado de cada conexión abierta.